Week 01 – Day 01

GenAI Basics – Answers

Test Case Generation Comparison using AI Tools

Scenario 1 – Search Functionality (ChatGPT)

Weak Prompt Analysis

Original Prompt: "Test the search feature."

Result / Issue: The request is ambiguous because it does not specify:

- What application is being tested
- What type of search (product search, customer search, global search, document search, etc.)
- Functional, performance, security, or usability scope
- Expected output format

Interpretation: A reasonable interpretation would be to generate generic search test scenarios:

Test Scenario
Verify search with valid input
Verify search with invalid input
Verify search with partial text
Verify search with exact match text
Verify search with uppercase characters
Verify search with lowercase characters
Verify search with special characters
Verify search with numeric values
Verify search with alphanumeric values

Test Scenario
Verify search with blank input
Verify search with leading spaces
Verify search with trailing spaces
Verify search with very long text
Verify search using Enter key
Verify search using Search button
Verify search result relevance
Verify no results found behavior
Verify search response time
Verify search results pagination
Verify search after page refresh

Optimized Prompt Framework

I – Instructions

- [MANDATORY] Generate test cases for Search Functionality in an E-commerce Website.
- [MANDATORY] Generate only 10 test cases.
- [MANDATORY] Include Functional and Non-Functional test cases.
- [MANDATORY] Include Positive and Negative test cases.
- [MANDATORY] Cover Search Suggestions, Search Results, Sorting, Filtering, and No Results scenarios.
- [MANDATORY] Cover Boundary Value validations for search input.
- [STRICT] Do not generate duplicate test cases.
- [STRICT] Use simple English.

C – Context

- Act as a Senior Test Engineer testing an E-commerce Web Application.
- Search should start after entering a minimum of 3 characters.
- Users should receive auto-suggestions while typing.
- Users should be able to sort results by Rating, Price, Most Ordered, and Latest Products.
- Users should be able to filter results by Category, Brand, Rating, and Price Range.
- If no products are found, the application should display an appropriate message.
- Search should be case-insensitive.

E – Example

TCID | Description | Priority | Preconditions | Steps | Expected Result

P – Persona

- Senior Test Engineer: For test case creation.
- Business Analyst: For validating business requirements.

O – Output

Generate output in a downloadable Excel format with columns:
TCID | Description | Test Type | Priority | Preconditions | Test Steps | Expected Result

T – Tone

Professional tone with clarity and structured formatting.

Generated Test Cases

TCID	Description	Test Type	Priority	Preconditions	Test Steps	Expected Result
TC_001	Verify search with valid product name (Positive)	Functional	High	User is on Home Page	1. Enter a valid product name with more than 3 characters. 2. Click Search.	Relevant products are displayed in search results.
TC_002	Verify search starts only after entering minimum 3 characters	Functional	High	User is on Home Page	1. Enter 1 character. 2. Enter 2 characters. 3. Observe search behavior.	Search is not triggered before entering 3 characters.
TC_003	Verify auto-suggestions are displayed while typing	Functional	High	User is on Home Page	1. Enter first 3 characters of a valid product. 2. Observe suggestions list.	Relevant auto-suggestions are displayed.
TC_004	Verify search is case-insensitive	Functional	Medium	Product exists in catalog	1. Search using uppercase product name. 2. Search using lowercase product name.	Same relevant results are displayed for both searches.
TC_005	Verify sorting search results by Price	Functional	High	Search results are available	1. Search for a product. 2. Select Sort By Price.	Results are displayed in correct price order.
TC_006	Verify filtering search results by Category and Brand	Functional	High	Search results are available	1. Search for a product. 2. Apply Category filter. 3. Apply Brand filter.	Results match the selected Category and Brand filters.

TCID	Description	Test Type	Priority	Preconditions	Test Steps	Expected Result
TC_007	Verify No Results scenario for invalid search keyword	Functional (Negative)	High	User is on Home Page	1. Enter a random keyword that does not exist. 2. Click Search.	Appropriate "No Products Found" message is displayed.
TC_008	Verify search with maximum allowed input length (Boundary Value)	Functional	Medium	User is on Home Page	1. Enter a search keyword with maximum allowed characters. 2. Click Search.	Search executes successfully without errors.
TC_009	Verify search with special characters only (Negative)	Functional	Medium	User is on Home Page	1. Enter special characters such as @#$%^. 2. Click Search.	System handles input gracefully and displays appropriate results or message.
TC_010	Verify search response time under normal load	Non-Functional (Performance)	High	Application is available	1. Enter a valid search keyword. 2. Perform search. 3. Measure response time.	Search results are displayed within the defined performance threshold.

Observations

When we gave ChatGPT a simple prompt like 'Test the search feature', it came back with very basic and generic results that weren't really useful for real testing.

Once we added more details about what we actually needed, like the app type, test coverage, and rules, the output was much more focused and ready to use.

Overall, the more context and clarity we gave, the better ChatGPT understood our requirements and delivered test cases that made sense for our project.

Scenario 2 – File Upload (Gemini)

Weak Prompt Analysis

Original Weak Prompt: "Generate test cases for file upload"

Expected Interpretation Scenarios:

- Valid Upload: Verify that a user can successfully upload a file with an allowed extension (e.g., .pdf, .jpg) within the accepted size limit.
- Invalid File Type: Attempt to upload a file with an unsupported extension (e.g., .exe, .bat) and verify the system rejects it with an appropriate error message.
- Maximum Size Boundary: Upload a file exactly at the maximum allowed size limit or 1 byte over limit.
- Zero Byte File: Attempt to upload an empty file (0 KB) to see how the system handles it.
- Multiple Files: Verify batch uploads up to the maximum batch limit.
- Duplicate File: Upload a file with the exact same name and extension.
- File Deletion: Verify that a user can remove or cancel a selected file before upload.
- UI & Usability: Drag and Drop, Browse Button, Progress Indicator, Success/Error States, File Preview.
- Security Testing: Extension Spoofing, Malicious Filenames (XSS/SQLi), Extremely Long Filenames, Bypass Client-Side Validation.
- Network & Edge Cases: Network Interruption, Resume Upload, Timeout Handling.

Optimized Prompt Framework

I – Instructions

- [MANDATORY] Generate test cases for File Upload functionality.
- [MANDATORY] Generate only 10 test cases.
- [MANDATORY] Include Functional and Non-Functional test cases.
- [MANDATORY] Include Positive and Negative test cases.
- [MANDATORY] Cover File Type Validation, File Size Validation, Upload Success, Replace File, and Delete File scenarios.
- [MANDATORY] Include Security-related validations.
- [STRICT] Do not generate duplicate test cases.
- [STRICT] Use simple English.

C – Context

- Act as a Senior Test Engineer testing a Document Management Application.
- Users should be able to upload PDF, JPG, PNG, and DOCX files.
- Maximum allowed file size is 10 MB.
- Users should receive a success message after upload.

- Users should be able to replace and delete uploaded files before submission.
- Unsupported file types should be rejected.
- Files exceeding the size limit should be rejected.

E – Example

TCID | Description | Priority | Preconditions | Steps | Expected Result

P – Persona

- Senior Test Engineer for test case generation.
- Test Lead for completeness review.

O – Output

Generate output in a downloadable Excel format with columns:
TCID | Description | Test Type | Priority | Preconditions | Test Steps | Expected Result

T – Tone

Professional tone with clarity and structured formatting.

Generated Test Cases

TCID	Description	Test Type	Priority	Preconditions	Test Steps	Expected Result
TC_001	Verify successful upload of standard allowed file formats	Functional (Positive)	High	User is on Document Upload panel	1. Select a valid .pdf or .jpg file under 10 MB. 2. Click upload.	File uploads successfully and human-readable success message appears.
TC_002	Verify upload rejection for unsupported malicious file formats	Functional (Negative)	High	User is on Document Upload panel	1. Select an unsupported file format like .exe or .bat. 2. Attempt upload.	The system rejects the document with an appropriate error message.
TC_003	Verify file size boundary limits validation at exactly 10 MB	Functional (Positive)	Medium	User is on Document Upload panel	1. Select a file that is exactly 10 MB in file size. 2. Execute upload.	The upload completes successfully without truncation or framework failure.

TCID	Description	Test Type	Priority	Preconditions	Test Steps	Expected Result
TC_004	Verify graceful rejection when file size exceeds the 10 MB limit	Functional (Negative)	High	User is on Document Upload panel	1. Choose a file size slightly over 10 MB. 2. Execute upload.	System flags file-size breach and rejects upload gracefully.
TC_005	Verify system handling when uploading an empty zero-byte file	Functional (Negative)	Medium	User is on Document Upload panel	1. Select an empty text/document file (0 KB). 2. Click upload.	System handles file gracefully, displaying error validation to user.
TC_006	Verify user's ability to replace a selected file before submission	Functional (Positive)	Medium	A file has been previously selected/uploaded	1. Click 'Replace File' option. 2. Choose a different valid file.	The original file is overwritten by the newly selected document.
TC_007	Verify file deletion or cancel options before final form submission	Functional (Positive)	High	A file has been selected in upload stage	1. Click 'Delete' or 'Cancel' option icon next to file name.	The selected file is removed cleanly from upload staging container.
TC_008	Verify backend-side defensive protection against extension spoofing	Non-Functional (Security)	High	User intercepts network upload parameter	1. Rename malicious script from malware.exe to malware.jpg. 2. Send upload.	Backend inspects binary file signatures/MIME type and blocks it.
TC_009	Verify filename sanitization against script injection payloads	Non-Functional (Security)	High	User is selecting a local file to upload	1. Name a valid file containing XSS payload: file'<script>alert(1)</script>.jpg.	The file name is completely sanitized or renamed upon server storage.
TC_010	Verify presence of a real-time progress indicator during file transfers	Non-Functional (Usability)	Medium	User has a large allowed document size	1. Trigger upload execution on a standard network connection.	A loading spinner, progress bar, or percentage tracker is visibly updating.

Observations

With a vague prompt, Gemini gave us a broad list of ideas but nothing we could actually sit down and test with — it felt more like brainstorming than real test cases.

When we told Gemini exactly what file types are allowed, what the size limit is, and what scenarios to cover, it gave us much cleaner and more usable results.

It was clear that Gemini performs better when it knows the context of the application — without that, it just guesses what we might need.

Scenario 3 – Shopping Cart (Copilot)

Weak Prompt Analysis

Original Weak Prompt: "Test the cart functionality."

Expected Interpretation Scenarios:

- Add to Cart: Single item, multiple items, same item multiple times, out-of-stock item.
- Remove from Cart: Single item, one quantity, remove all items.
- Update Quantity: Increase, decrease, set to zero, invalid quantity.
- Price Calculation: Price per item, subtotal, tax/discount/shipping, final total formula, rounding.
- Persistence: Refresh page, logout/login, close browser.
- UI/UX Validation: Cart icon count, item details display, responsive design, empty cart message.
- Navigation: Click item, continue shopping button, proceed to checkout.
- Edge Cases: Maximum quantity, rapid multiple clicks, network interruption, large number of items.
- Performance: Load time with many items, smooth updates.
- Security: Price manipulation check, cart data integrity, unauthorized cart access.
- Compatibility: Browsers (Chrome, Edge, Firefox, Safari) and Devices.

Optimized Prompt Framework

I – Instructions

- [MANDATORY] Generate test cases for Shopping Cart functionality.
- [MANDATORY] Generate only 10 test cases.
- [MANDATORY] Include Functional and Non-Functional test cases.
- [MANDATORY] Include Positive and Negative test cases.
- [MANDATORY] Cover Add to Cart, Update Quantity, Remove Product, Cart Calculation, and Empty Cart scenarios.
- [MANDATORY] Cover Discount and Coupon validation scenarios.
- [STRICT] Do not generate duplicate test cases.
- [STRICT] Use simple English.

C – Context

- Act as a Senior Test Engineer testing an E-commerce Shopping Cart module.
- Users should be able to add products to the cart.
- Users should be able to update product quantities.
- Users should be able to remove products from the cart.
- Cart total should update automatically.
- Available discounts and coupons should be applied correctly.
- Product quantity should not exceed available stock.

- If the cart is empty, an appropriate message should be displayed.

E – Example

TCID | Description | Priority | Preconditions | Steps | Expected Result

P – Persona

- Senior Test Engineer for test case generation.
- Test Lead for quality review.
- Business Analyst for validating cart business rules.

O – Output

Generate output in a downloadable Excel format with columns:
TCID | Description | Test Type | Priority | Preconditions | Test Steps | Expected Result

T – Tone

Professional tone with clarity and structured formatting.

Generated Test Cases

TCID	Description	Test Type	Priority	Preconditions	Test Steps	Expected Result
TC_001	Verify adding a single valid item to the cart	Functional (Positive)	High	User is on Product Detail Page	1. Click 'Add to Cart' button for a valid in-stock product.	Item appears in the cart with correct name, price, and a quantity of 1.
TC_002	Verify adding the same item multiple times increments quantity	Functional (Positive)	High	Product already exists in the cart	1. Navigate to the same product page. 2. Click 'Add to Cart' again.	Quantity increments correctly without creating duplicate list entries.
TC_003	Verify out-of-stock items cannot be added to the cart	Functional (Negative)	High	Product is marked as Out of Stock	1. Try to add the out-of-stock item to the cart.	Proper restriction error message is displayed; item is not added.
TC_004	Verify increasing item quantity updates total price automatically	Functional (Positive)	High	Items exist inside the shopping cart	1. Open cart page. 2. Click '+' button to increase quantity.	Quantity increases, and subtotal/final totals update dynamically.

TCID	Description	Test Type	Priority	Preconditions	Test Steps	Expected Result
TC_005	Verify entering an invalid or negative quantity displays validation error	Functional (Negative)	Medium	Items exist inside the shopping cart	1. Enter a negative value or non-numeric text in quantity field.	System rejects input and displays appropriate validation error.
TC_006	Verify removing a single item updates the cart layout successfully	Functional (Positive)	High	Product exists inside the cart	1. Click the 'Remove' or trash bin icon next to an item.	The selected item completely disappears from the cart listing.
TC_007	Verify mathematical subtotal, tax, and final total calculations	Functional (Positive)	High	Multiple items are present in cart	1. Inspect subtotal, tax, shipping charges, and final grand total.	Grand total precisely equals subtotal + tax + shipping - discounts.
TC_008	Verify successful application of a valid coupon code discount	Functional (Positive)	High	User is on the Cart page with items	1. Input a valid active discount coupon. 2. Click apply.	Discount is deducted correctly and final price updates automatically.
TC_009	Verify user interface display layout when the cart is completely empty	Functional (Positive)	Medium	Shopping cart is currently empty	1. Navigate directly to the Cart page view.	An appropriate message like 'Your cart is empty' is displayed clearly.
TC_010	Verify protection against price manipulation via UI or API requests	Non-Functional (Security)	High	User is authenticated on checkout step	1. Intercept cart request and alter item price parameter.	The backend identifies validation discrepancy and rejects the manipulated request.

Observations

When we asked Copilot to 'test the cart functionality', it tried its best but ended up covering too many things at a surface level without going deep into any of them.

After we refined the prompt with specific scenarios like coupon validation, quantity updates, and price calculations, Copilot gave us test cases that actually matched what we wanted to verify.

The experience showed us that Copilot is quite capable when guided well — it just needs us to be specific about what matters most in our application.